Now our design is in much better shape. By splitting the functionality across more specialized classes, we have reduced the impact of any change requests. Finance and warehouse management can both ask for change (concurrently, even), and it is easier to manage. This is because – with the new design – different components are servicing the needs of different groups of stakeholders, and the scope of each change becomes smaller: there's less code to modify (always a good thing).

Open-Closed Principle (OCP)

The term "open-closed principle" is credited to Bertrand Meyer, from his 1988 book *Object-Oriented Software Construction*. OCP states that modules should be open for extension but closed for modification. A well-designed class should not need to be modified if extra functionality is required; instead, the extra functionality should be provided by extending the class or by composing objects (building the functionality of a run-time object by adding other objects that define its behavior – we'll see this in action when we look at design patterns later on).

Let's see how inheritance-based OCP solves problems. A common "code-smell" (a sign that the design can be improved) is the use of logic to determine what an object represents in the problem domain. This can happen if a class is too generalized to accurately represent any particular problem domain entity.

```
class Vehicle
{
    public function __construct(private string $type)
    {
    }

    public function getMaxPassengers(): int
    {
        switch ($this->type) {
            case 'car':
                return 4;
                break;
            case 'bus':
                return 30;
                break;
```